

# Phân Tích Phê Bình về SDD

Ưu, Nhược và Ranh Giới — The Human in the Loop

PLAYBOOK SDD + ADD · CHƯƠNG 8



# Tại sao cần một chương phê bình?

## LỜI MỞ ĐẦU

Bảy chương trước đã xây dựng SDD từ nền tảng triết lý đến công cụ thực chiến. Nhưng một cuốn sách trung thực không thể kết thúc ở đó. Mọi phương pháp, dù hiệu quả đến đâu, đều có giới hạn — và người dùng thông minh là người biết cả hai: khi nào nên áp dụng và khi nào nên đặt xuống.

Chương này cố tình mang tông giọng khác. Không phải hướng dẫn, không phải tutorial — mà là đối thoại. Chúng ta sẽ nhìn SDD qua mắt những người phê bình nó, qua dữ liệu thực tế, và qua những câu hỏi khó nhất.

- ❏ **Điểm nhìn trung lập:** Chương này không cố thuyết phục bạn SDD là câu trả lời cho mọi thứ. Nó cũng không cố thuyết phục bạn ngược lại. Mục tiêu: trang bị đủ góc nhìn để bạn tự đưa ra phán xét cho hoàn cảnh của mình.

"Công cụ tốt nhất là công cụ bạn hiểu đủ sâu để biết khi nào KHÔNG dùng nó."

# Nội dung Chương 8

MỤC LỤC

|                                                          |                                                                    |                                        |
|----------------------------------------------------------|--------------------------------------------------------------------|----------------------------------------|
| 01                                                       | 02                                                                 | 03                                     |
| <b>8.1 — Điểm mạnh thực sự của SDD</b>                   | <b>8.2 — Những chỉ trích hợp lý</b>                                | <b>8.3 — Khi nào SDD KHÔNG phù hợp</b> |
| Reproducibility · Quality Gates · Knowledge · Speed      | Waterfall · Context Blindness · Over-spec · Overhead               | R&D · Ma trận Project Type × Team Size |
| 04                                                       | 05                                                                 |                                        |
| <b>8.4 — Góc nhìn đa chiều — SDD trong mắt cộng đồng</b> | <b>8.5 — "The Cost of a Bad Spec" — Khi Spec sai nhưng AI đúng</b> |                                        |
| Thoughtworks · GitHub · Marmelab · Pragmatic SDD         | Anatomy · Case Study · GIGO · Defensive Writing · Fix vs Rewrite   |                                        |



#### SECTION 8.1

## Điểm mạnh thực sự của SDD

Những lợi ích có thể đo lường được, có dữ liệu thực tế, và quan trọng hơn — có cơ chế giải thích rõ ràng tại sao chúng xảy ra

# Kết quả nhất quán từ cùng một Spec

Đây là lợi ích ít được nói đến nhất nhưng có giá trị kỹ thuật cao nhất. Khi có SPEC.md rõ ràng, hai developer khác nhau (hoặc hai AI sessions khác nhau) implement cùng feature sẽ tạo ra code có behavior giống nhau — ngay cả khi implementation details khác nhau.

## Reproducibility — Không có SDD vs Có SDD

```
# KHÔNG có SDD:
# Dev A: "Validate email" → kiểm tra format + DNS lookup
# Dev B: "Validate email" → chỉ kiểm tra có @ hay không
# AI session 1: regex /^[^@]+@[^@]+\.[^@]+$ /
# AI session 2: dùng thư viện validator.js
# → 4 cách implement, 4 behavior khác nhau, 4 test expectation khác nhau

# CÓ SDD (SPEC.md §3):
# "WHEN user submits email, THE system SHALL validate:
# 1. Format: regex RFC 5322 compliant
# 2. Domain: MX record lookup (async)
# 3. Disposition: not in blacklist (sync)
# Response time: < 500ms total"
# → Mọi implementation đều phải thỏa 3 điều kiện này
# → Testable, verifiable, reproducible
```

📌 **Key Insight:** Spec là "bản hợp đồng" đảm bảo behavior — không phải implementation style. Điều này đặc biệt quan trọng khi AI models thay đổi hoặc khi cần onboard người mới.

# Lỗi bị chặn ở tầng Spec, không phải tầng Code

IBM Systems Sciences Institute: chi phí sửa lỗi tăng exponentially theo giai đoạn phát hiện. SDD đẩy quality check về giai đoạn sớm nhất — giảm cost of bug fix tối đa 1000x.

| Giai đoạn phát hiện   | Relative cost | SDD giảm tần suất?      | Cơ chế                           |
|-----------------------|---------------|-------------------------|----------------------------------|
| Requirements / Spec   | 1x            | ✔ Tăng (AI Review)      | Clarification Trigger tìm gaps   |
| Design / Architecture | 3–6x          | ✔ Tăng (PLAN.md review) | AI hỏi Questions for Human       |
| Coding                | 10x           | ⚠ Giảm một phần         | EARS tags buộc align với spec    |
| Testing               | 15–40x        | ✔ Giảm đáng kể          | Acceptance Criteria → test cases |
| Production            | 40–1,000x     | ✔ Giảm nhiều            | Constitution gates + Validation  |

## McKinsey 2025 — Bằng chứng từ Industry

**20–45%**

Giảm defect density

**75–85%**

Sprint delivery rate (vs 40–60%)

**30–40%**

Giảm AI code cần rework (GitHub)

"Teams using structured specification approaches before AI code generation reported 20–45% reduction in defect density compared to prompt-only workflows. The primary mechanism: errors caught at spec review stage, not testing stage." — McKinsey, "The State of AI in Software Development", 2025

**⚠ Lưu ý quan trọng:** Con số 20–45% có range rộng vì phụ thuộc nhiều vào quality của spec. Bad spec không chỉ không giúp ích — nó có thể tệ hơn không có spec.

# Ngôn ngữ chung cho cả team

SPEC.md tạo ra một source of truth duy nhất mà mọi người — từ PM đến dev đến AI — đều làm việc từ đó. Khi có tranh luận về behavior, câu trả lời không phải "tôi nghĩ là..." mà là "spec nói gì?". Đây là sự dịch chuyển quan trọng từ opinion-driven sang evidence-driven development.

# KHÔNG có SDD — Sprint planning meeting:

# PM: "Feature này đơn giản thôi, 2 ngày là xong"

# Dev: [nghĩ về 10 edge cases chưa được nói đến] "Hmm..."

# QA: [chưa biết **test** cái gì] "OK"

# → Mỗi người có mental model khác nhau. Conflict khi demo.

# CÓ SDD — Sprint planning với SPEC.md v1.0.0 đã approved:

# PM: "Feature này có 8 EARS requirements và 7 acceptance criteria"

# Dev: "Task breakdown đã có, T004 cần clarify thêm Q2"

# QA: "Tôi sẽ test theo acceptance criteria, đặc biệt 3 Unwanted patterns"

# → Cùng mental model. Ít surprise. Ít rework.

# Measurable: team velocity variance

# Không có SDD: sprint delivery rate varies 40–60% of commitments

# Có SDD: sprint delivery rate improves to 75–85% (GitHub, 2024)



# Tri thức không rời đi cùng con người

📌 **Business Value:** Theo LinkedIn 2024: average tenure trong tech là 2.1 năm. Dự án trung bình kéo dài 3–5 năm. Trung bình mỗi dự án thay đổi 100% team. SDD đảm bảo knowledge tồn tại độc lập với con người — đây là competitive advantage không nhìn thấy được nhưng rất thực.

| Kịch bản                 | Không có SDD                                      | Có SDD                                                       |
|--------------------------|---------------------------------------------------|--------------------------------------------------------------|
| Dev senior nghỉ việc     | Team mất 3–6 tháng để "rediscover" business logic | SPEC.md chứa reasoning + edge cases.<br>Onboard trong 1 tuần |
| Onboard junior mới       | Học bằng cách "hỏi senior" và "đọc code"          | Đọc SPEC.md hierarchy, hiểu context trong 2 ngày             |
| Bug report từ production | "Tại sao code này làm vậy?" — không ai nhớ        | Trace EARS tag → spec section → business decision            |
| Compliance audit         | "Hệ thống xử lý PII như thế nào?" — phải đọc code | data-governance.md → đọc spec → trả lời trong giờ            |
| AI model thay đổi        | Mỗi session mới AI lại "đoán" context             | SPEC.md + CONTEXT.md = AI luôn có đủ context                 |

# Song song hóa nhờ Spec

Một trong những lợi ích ít hiển nhiên nhất của SDD là khả năng parallel implementation — chạy nhiều việc cùng lúc khi spec đã được chốt.

```
# TRADITIONAL (sequential):
# Week 1: Dev A viết CartService
# Week 2: Dev B viết CartRouter (chờ A xong)
# Week 3: Dev C viết CartTests (chờ B xong)
# Week 4: Integration + fixes
# Total: 4 weeks

# SDD (parallel — spec locked TRƯỚC khi ai bắt đầu code):
# Week 1 (parallel):
# Dev A: CartService (theo SPEC.md §3, PLAN.md components)
# Dev B: CartRouter (theo API contract trong PLAN.md)
# Dev C: CartTests (theo Acceptance Criteria trong SPEC.md §7)
# AI: DB migrations (theo Data section SPEC.md §5)
#
# Integration ít friction vì:
# - Dev A và B đồng thuận về interface qua PLAN.md
# - Dev C viết tests từ spec, không từ implementation
# Total: 1.5 weeks (vs 4 weeks) cho cùng feature

#
```

# 5 Lợi ích Thực chất của SDD

## 8.1 — TỔNG KẾT ĐIỂM MẠNH

### Reproducibility

Spec là "bản hợp đồng" behavior — mọi implementation phải thỏa mãn cùng requirements. Không phụ thuộc vào người làm hay AI model nào.

### Quality Gates sớm hơn

Lỗi bị chặn ở tầng spec (cost 1×) thay vì production (cost 1,000×). McKinsey: 20–45% giảm defect density.

### Team Alignment

Single source of truth. Từ opinion-driven sang evidence-driven. Sprint delivery: 40–60% → 75–85%.

### Knowledge Persistence

Tri thức không rời đi khi nhân sự thay đổi. Onboard từ tháng xuống tuần. Business logic có thể audit.

### Time-to-Market

Parallel implementation khi spec locked. Feature 4 tuần sequential → 1.5 tuần parallel. Spec đủ rõ = mọi thành phần được build độc lập.

SECTION 8.2

## Những chỉ trích hợp lý

Nhìn thẳng vào điểm yếu — những chỉ trích đến từ người đã dùng SDD, thấy vấn đề, và phản biện có cơ sở



## "SDD là Waterfall trá hình"

Nguồn: Marmelab blog "Spec-Driven Development — The New Waterfall?" (2024). Lập luận: SDD yêu cầu viết spec đầy đủ trước khi code, giống hệt Waterfall. Chỉ khác là agent thực hiện thay vì developer.

"SDD replicates the fundamental assumption of Waterfall: that you can know what you need before you build it. History has shown this to be false for most software. Requirements change. Users surprise you. The market shifts. Locking specs before coding is optimism masquerading as rigor." — Marmelab, 2024

 **Phân tích:** Marmelab đúng một phần: mô tả đúng SDD khi bị áp dụng sai — chốt spec toàn bộ dự án trước khi code. Đó thực sự là Waterfall với tên mới. Nhưng đây là cách dùng SAI SDD, không phải cách SDD được thiết kế.

# Waterfall vs "SDD Sai" vs "SDD Đúng"

|                     | Waterfall                    | "SDD sai cách"                | "SDD đúng cách" (Spec-per-Feature)      |
|---------------------|------------------------------|-------------------------------|-----------------------------------------|
| Spec scope          | Toàn bộ dự án trước khi code | Module toàn bộ trước khi code | Chỉ feature đang build trong sprint này |
| Spec timing         | 6 tháng đầu dự án            | Trước khi team bắt đầu sprint | Ngày 1 của sprint, chỉ feature này      |
| Spec immutability   | Locked vĩnh viễn             | Locked trong project          | Locked trong sprint (2 tuần)            |
| Respond to change   | Khó — change request process | Khó — renegotiate spec        | Dễ — spec thay đổi theo sprint          |
| Agile compatibility | ❌ Không tương thích          | ⚠️ Căng thẳng                 | ✅ Tương thích hoàn toàn                 |

👉 **Phản biện — "Spec-per-Feature là Agile trên Steroid":** Khi áp dụng đúng nguyên lý Spec-per-Feature, SDD không mâu thuẫn với Agile — nó tăng cường Agile. Không có "frozen requirements" — chỉ có "frozen spec cho sprint này". Spec thay đổi theo sprint, không theo dự án.

# Spec-per-Feature trong Sprint

# SPRINT PLANNING (Thứ 2):

# Backlog item: "As a user, I want to save multiple addresses"

# → Pha 0: Context Discovery (2h)

# → Pha 1: Write SPEC.md draft → AI review → Lock spec (2h + 30min + 30min)

# → Pha 2–3: PLAN.md + TASKS.md (1h via AI)

# → Total spec overhead: ~6h = 1 working day

# SPRINT EXECUTION (Thứ 3 → Thứ 6):

# 4 ngày implement từ locked spec + tasks

# Không có ambiguity. Không có "what did you mean by X?"

# SPRINT REVIEW (Thứ 6):

# Demo dựa trên acceptance criteria từ spec

# "Test case 3: cart merge đúng không?" → Demo → Verify

# ĐIỀU CHỈNH CHO SPRINT TIẾP THEO:

# Business thay đổi requirement → Spec MỚI cho sprint tới

# Không có "frozen requirements" — chỉ có "frozen spec cho sprint này"

# → Đây KHÔNG phải Waterfall. Đây là Agile với higher quality gate.

## Context Blindness — mù ngữ cảnh

Đây là chỉ trích kỹ thuật quan trọng nhất và ít có phản biện dễ dàng. AI chỉ biết những gì được viết trong spec. Nếu spec thiếu ràng buộc ngầm định — thường là những thứ mọi developer trong team "đương nhiên biết" — AI sẽ tạo ra code đúng theo spec nhưng sai theo thực tế.

 **Ví dụ thực tế — Context Blindness:** Spec nói: "cache product data với TTL 5 phút". AI implement Redis cache hoàn hảo theo spec. Vấn đề: Legacy system dùng custom event bus không broadcast cache invalidation — spec không đề cập vì team "đương nhiên biết". Kết quả: Stale data bug xuất hiện sau 5 phút mỗi product update.

Có những loại context mà con người biết một cách hiển nhiên nhưng rất khó verbalize:

- Tacit knowledge: "Hệ thống này khi load cao thì X thường xảy ra"
- Historical context: "Module Y bị vá lỗi như vậy vì incident năm ngoái"
- Environmental quirks: "Redis cluster này có latency spike vào 2–4am"
- Implicit performance model: "Query này đủ fast với 10k rows nhưng fail với 1M"



# Quản lý, không giải quyết hoàn toàn

| Loại context bị thiếu      | Giảm thiểu SDD                          | Không giải quyết được                         |
|----------------------------|-----------------------------------------|-----------------------------------------------|
| Explicit constraints thiếu | CONTEXT.md section, Clarification-First | Constraints bạn không biết là mình không biết |
| Legacy system quirks       | Document trong constitution.md          | Undocumented behaviors chưa được khám phá     |
| Performance context        | Non-functional requirements với numbers | Real-world load patterns khó spec trước       |
| Org knowledge              | Context Document + stakeholder input    | Political context, unstated priorities        |

 **Kết luận trung thực:** SDD làm giảm context blindness đáng kể nhưng không loại bỏ hoàn toàn. Đây là lý do Human-in-the-Loop không phải là tính năng optional của SDD — đó là thành phần bắt buộc. AI generate code, nhưng con người với full context phải review mọi output.

# Over-specification — Analysis Paralysis

Chỉ trích này đến từ những engineer có kinh nghiệm với "analysis paralysis": team dành nhiều thời gian viết spec hơn là implement. Khi mọi người mới học SDD, xu hướng tự nhiên là "viết càng chi tiết càng tốt" — điều này phản tác dụng.

## Dấu hiệu Over-specification

- SPEC.md > 500 dòng cho một feature sprint-size
- Spec describe implementation details thay vì behavior ("dùng Redis hash" thay vì "cache product")
- Team mất > 30% sprint time để viết và review spec
- AI generate code hoàn toàn đúng spec nhưng code không thể maintain được

✓ **Giải pháp:** Risk Matrix từ Chương 5 (Sketch / Detailed / Formal) là câu trả lời. Không phải mọi thứ cần cùng level of detail. Spec behavior, không phải implementation.

# Overhead không tương xứng cho dự án nhỏ

Một developer làm một website cá nhân trong cuối tuần không cần 8-component SPEC.md với EARS notation. Overhead của full SDD chỉ justify được khi scale đủ lớn.

| Project type         | Full SDD overhead   | Benefit                 | Kết luận                    |
|----------------------|---------------------|-------------------------|-----------------------------|
| 1-person, < 1 tuần   | 6h/feature cho spec | ~2h code faster         | ROI âm — đừng dùng          |
| 2-3 người, 1 tháng   | 6h/feature cho spec | ~10h saved in rework    | ROI dương, tùy loại feature |
| 5+ người, 6+ tháng   | 6h/feature cho spec | ~40h saved per feature  | ROI rõ ràng, nên dùng       |
| Enterprise, 2+ years | 6h/feature cho spec | Hundreds of hours saved | Không dùng là sai sót lớn   |

 **Kết luận:** Chỉ trích về overhead là đúng trong context phù hợp. SDD không được thiết kế cho mọi loại dự án — phần 8.3 sẽ đi sâu hơn về khi nào KHÔNG dùng SDD.



SECTION 8.3

# Khi nào SDD KHÔNG phù hợp

Biết khi nào dừng lại quan trọng không kém biết khi nào bắt đầu

# Khi bạn không biết mình muốn gì

SDD giả định bạn có thể define "Cái gì" trước khi AI implement "Như thế nào". Nhưng có những loại công việc mà "cái gì" chính là điều bạn đang cố tìm ra. Yêu cầu viết Spec trước là yêu cầu biết đáp án trước khi đặt câu hỏi.

## Khi Spec triệt tiêu sáng tạo

- "The fastest spec is no spec" — đúng trong context exploration
- Khi prototype UX, mỗi lần dùng thử có thể đảo ngược toàn bộ assumption
- Nếu có SPEC.md, thay đổi trở thành "spec violation" thay vì "learning"
- Trong R&D, sai là thông tin quý giá. Locked spec biến sai thành nợ.

## Nhận biết khi nào là Exploration:

- Không có precedent — bạn không biết user sẽ tương tác thế nào
- Hypothesis-driven — mục tiêu là validate giả định, không phải implement requirement
- Time-boxed throwaway — kết quả là learning, không phải production code
- Metric còn chưa defined — bạn chưa biết "thành công" trông như thế nào

# EXPERIMENT.md — Thay thế SPEC.md cho R&D

## ## Hypothesis

Chúng tôi tin rằng [hành vi X] sẽ dẫn đến [outcome Y].

## ## What we are testing

[Mô tả rõ cái gì được test trong 1-2 câu]

## ## Success metric

[Cách đo: "user completes flow in < 30 seconds without assistance"]

## ## Time box

[2 days max — nếu quá thì stop, không phải extend]

## ## NOT a spec

Đây không phải spec. Code từ experiment này sẽ được THROW AWAY.

Nếu hypothesis validate → viết SPEC.md thực sự.

Nếu hypothesis fail → document learning, move on.

## # Cách dùng với AI:

# "Build a quick prototype to test this hypothesis.

# Code không cần clean. Không cần tests.

# Focus vào making the interaction visible."

# → AI biết đây là throwaway, không over-engineer

❏ **Key Distinction:** EXPERIMENT.md không phải là spec nhẹ hơn — đây là document với mục đích khác hoàn toàn. Spec mô tả requirements. Experiment mô tả hypothesis. Spec tạo ra production code. Experiment tạo ra learning.

# Hướng dẫn thực tế — Khi nào dùng SDD

| Loại dự án                         | SDD Score   | SDD Level        | Ghi chú                                            |
|------------------------------------|-------------|------------------|----------------------------------------------------|
| Hệ thống Tài chính / ERP / Banking | 10/10 ★★★★★ | Formal (Mức 3)   | Cần chính xác tuyệt đối. Mọi edge case phải spec.  |
| Refactor Legacy Code               | 9/10 ★★★★★½ | Detailed         | Spec đảm bảo không mất behavior cũ khi refactor.   |
| API / Backend Service (production) | 9/10 ★★★★★½ | Detailed         | Nhiều consumer phụ thuộc → contract stability.     |
| SaaS Product (B2B)                 | 8/10 ★★★★★  | Detailed/Formal  | Customer SLA, compliance, reliability.             |
| Mobile App (complex)               | 7/10 ★★★★★½ | Detailed         | UX may evolve, nhưng business logic cần spec.      |
| Internal Tools / Admin panels      | 5/10 ★★★½   | Sketch/Detailed  | Balance: cần đủ spec nhưng tránh over-engineer.    |
| Game Prototyping                   | 4/10 ★★     | Sketch chỉ       | Game feel requires iteration. Spec sẽ gây cản trở. |
| Landing Page / Marketing site      | 2/10 ★      | Prompt trực tiếp | Over-engineering. Prompt + iterate nhanh hơn.      |
| Personal project < 1 tuần          | 1/10 ★      | Không cần        | You are spec. Overhead > benefit.                  |
| Hackathon (< 48h)                  | 0/10 ×      | Skip hoàn toàn   | Time is the constraint. Ship anything that works.  |

## 3 Câu hỏi trước khi bắt đầu

Nếu không chắc chắn, hãy đặt 3 câu hỏi này:

1

### Câu hỏi 1

Nếu AI implement sai, tôi có phát hiện ra ngay không?

**Không phát hiện ngay = cần spec**

2

### Câu hỏi 2

Có người khác (dev, QA, AI session khác) cần hiểu cùng context này không?

**Có = cần spec**

3

### Câu hỏi 3

Feature này sẽ được maintain hơn 3 tháng không?

**Có = cần spec**

 **Quy tắc:** Trả lời "không" cho cả 3 câu → bỏ qua SDD, dùng prompt trực tiếp. Trả lời "có" cho  $\geq 2$  câu → dùng ít nhất Sketch level.





#### SECTION 8.4

## Góc nhìn đa chiều — SDD trong mắt cộng đồng

SDD trong mắt cộng đồng — Thoughtworks, GitHub, Marmelab, Indie Developers, và Pragmatic SDD

# Thoughtworks & GitHub Research

## Thoughtworks Technology Radar 2025 — "Ứng dụng có điều kiện"

Đặt Specification-Driven approaches vào category "Adopt" nhưng với caveats rõ ràng. SDD có giá trị cao nhất khi AI được dùng như một "pair programmer" review spec trước khi implement.

"The spec is a living document, not a contract. Teams that treat it as immutable are setting themselves up for the same problems as waterfall. Teams that use it as a communication tool and quality gate are seeing real benefits."

Bài học: Spec là công cụ giao tiếp. Khi nó trở thành bureaucracy, nó mất đi giá trị.

## GitHub (Microsoft Research) — "Spec Kit như first-class citizen"

"We see Spec Kit as the missing layer between user stories and code. Teams that adopt it see 30–40% reduction in AI-generated code that needs significant rework." — GitHub Engineering Blog, 2024

Bài học: Spec là code artifact, không phải documentation. Treat it accordingly.

# Marmelab & Indie Developer Community

## Marmelab — "Phản biện chính thống"

"SDD works beautifully for teams building financial systems or regulated software. For a startup trying to find product-market fit, it's the wrong tool at the wrong time."

Lập luận core: complexity của SDD không scale down — designed cho enterprise nhưng được marketing cho mọi người. Nhiều indie dev và startup áp dụng sai context, gặp overhead mà không có benefit tương ứng.

Bài học: Sự phê bình của Marmelab thực ra là về việc áp dụng sai context, không phải về SDD bản thân nó.

## Indie Developer Community — "Pha trộn thực dụng"

"I don't do full SDD but I always write the EARS requirements before giving anything to Claude. That one habit alone cut my rework by half." — dev.to, Hacker News (paraphrased)

Bài học: SDD không phải all-or-nothing. Các thành phần riêng lẻ có giá trị độc lập.

# Pragmatic SDD — Một cách tiếp cận mới

## Pragmatic SDD — 3 Nguyên tắc

- Spec cho những gì quan trọng. Không phải mọi thứ cần spec bằng nhau. Module rủi ro cao → Detailed/Formal. CRUD boilerplate → Sketch hoặc prompt trực tiếp.
- EARS bất kể level. Dù Sketch hay Formal, dùng EARS cho requirements. Đây là component ROI cao nhất của SDD. Standalone. Không cần toàn bộ framework.
- Clarification trước mọi thứ phức tạp. Trước khi AI implement bất cứ điều gì có >5 business rules hoặc >3 edge cases, chạy Clarification Trigger.

— Đây là task gì? —

Hackathon / Prototype / "Tôi chưa biết mình muốn gì" |  
→ SKIP SPEC. Prompt trực tiếp. Iterate. |

Landing page / Marketing copy / UI chỉnh sửa màu sắc |  
→ SKIP SPEC. Prompt + in-context instruction. |

Utility function đơn giản (< 20 lines, 0-1 edge cases) |  
→ SKETCH: 5-10 dòng, không cần EARS. |

Feature với 3-10 business rules, rủi ro vừa |  
→ DETAILED SPEC: 8 thành phần + EARS + Clarification |

Payment / Auth / Compliance / Legacy refactor |  
→ FORMAL SPEC: Detailed + State Diagram + Constitution |

## # Luôn làm bất kể level:

#



#### SECTION 8.5

## "The Cost of a Bad Spec" — Khi Spec sai nhưng AI đúng

Nguy hiểm nhất không phải là khi bạn KHÔNG có spec — mà là khi bạn CÓ spec sai nhưng không biết nó sai

# Spec sai nhưng nghe có vẻ đúng

Đây là cảnh báo quan trọng nhất trong toàn bộ cuốn sách. Và nó đối lập hoàn toàn với những gì bạn có thể expect:

## Developer không có Spec

- Code cẩn thận
- Hỏi nhiều câu hỏi
- Deliver dần từng phần
- Tự nhận ra khi không chắc

## AI với Bad Spec

- Code tự tin
- Không hỏi gì cả
- Deliver 1,000 dòng code sai một cách hoàn hảo
- False confidence không có giới hạn

"Đây là thứ chúng ta cần sợ hơn là 'AI không có spec'."

# 5 Loại Bad Spec thường gặp

| Loại Bad Spec           | Biểu hiện                                                             | AI behavior                                | Hậu quả                                        |
|-------------------------|-----------------------------------------------------------------------|--------------------------------------------|------------------------------------------------|
| Internally inconsistent | Section 3 nói "max 10 items" nhưng Section 7 test case "add 15 items" | Chọn một version, bỏ qua cái còn lại       | Feature pass spec review nhưng fail production |
| Domain error            | "Payment confirmed" → "ship order" (thiếu bước inventory check)       | Implement đúng spec → code sai business    | Oversell, lost orders, angry customers         |
| Ambiguous threshold     | "Validate email format" mà không define "format"                      | Pick một cách, thường là không đủ strict   | Invalid emails pass, valid emails blocked      |
| Missing invariant       | Không mention rằng user_id không thể thay đổi sau create              | Implement update endpoint allow all fields | Security vulnerability, data corruption        |
| Wrong actor             | Spec nói "user can approve" nhưng thực ra chỉ admin được approve      | Implement với wrong role check             | Privilege escalation bug                       |



# 1,000 dòng code sai hoàn hảo — Loan Repayment

## Bối cảnh

# Fintech startup. Feature: Loan repayment processing.  
# PM viết spec 3 trang, nhìn đầy đủ, có EARS notation.  
# Spec approved. AI (Cline) implement. 1,200 dòng code.  
# Code review pass. Tests green. Deploy to staging.

## The Bad Spec (excerpt):

# SPEC §3: Functional  
# WHEN borrower makes payment, THE system SHALL:  
# 1. Deduct amount from outstanding\_balance  
# 2. Update payment\_status to "completed"  
# 3. Generate receipt  
# 4. Notify borrower via email  
# → Looks correct, right?

## What was missing (không ai nghĩ đến):

# Thiếu 1: Payment allocation logic  
# Loan có principal + interest + fees. Khi borrower trả 500K:  
# Trả fees trước? Hay principal? Hay pro-rata?  
# Spec chỉ nói "deduct from outstanding\_balance" (một số đơn giản)  
# AI implement: deduct từ principal (vì đó là số lớn nhất)  
# Business rule thực: fees first, then interest, then principal  
# → Mọi payment trong staging đều allocated sai

# Thiếu 2: Concurrent payment protection

# Spec không mention idempotency  
# User double-click "Pay" → 2 payments submitted simultaneously  
# → Race condition: both pass "check balance >= amount"

# Thiếu 3: Partial payment behavior

# Nếu amount > outstanding\_balance, xử lý thế nào?  
# AI implement: allow overpayment (generate negative balance)  
# Business rule: reject với message "amount exceeds outstanding"



## Phát hiện sau 3 ngày, Rewrite toàn bộ

3

Ngày phát hiện trong staging

1,200

Dòng code phải rewrite (không phải fix)

2W

Delay cho launch

\$15K

Cost estimate (dev time + delay)

⊗ **Root Cause:** Spec viết đúng syntax, đúng format, nhưng sai domain. AI không có domain knowledge để phát hiện thiếu sót. Human reviewer không catch vì "spec nhìn có vẻ đủ". → Đây là lý do Domain Expert Walk-through là bắt buộc cho high-risk spec.

# Nguyên lý bất biến của SDD

GIGO trong SDD context

$f(\text{spec}) = \text{code}$

$\text{quality}(\text{code}) \leq \text{quality}(\text{spec})$

# Cụ thể hơn:

# Good Spec + Good AI  $\rightarrow$  Good Code

# Good Spec + Bad AI  $\rightarrow$  Mediocre Code (still better than no spec)

# Bad Spec + Good AI  $\rightarrow$  Bad Code (executed PERFECTLY — nguy hiểm nhất)

# Bad Spec + Bad AI  $\rightarrow$  Very Bad Code

# No Spec + Good AI  $\rightarrow$  Unpredictable Code

# Kết luận ngược chiều:

# Cải thiện AI model ít quan trọng hơn cải thiện Spec quality.

# Nếu spec quality không thay đổi:

# dùng model tốt hơn  $2\times$  = code tốt hơn  $\sim 5\%$

# Nếu spec quality tăng gấp đôi:

# dùng model cũ = code tốt hơn  $\sim 80\%$

#  $\rightarrow$  Đầu tư vào KỸ NĂNG VIẾT SPEC là ROI cao hơn

# đầu tư vào AI model tốt hơn

 **Takeaway:** Thay vì hỏi "AI nào tốt nhất?", hãy hỏi "Spec của tôi có đủ tốt không?". Đó là đòn bẩy có ROI cao hơn nhiều.

## 4 Kỹ thuật — Viết spec như không tin mình

1

### Kỹ thuật 1 — Devil's Advocate Review

Đọc spec với tư cách là người muốn implement sai mà vẫn đúng spec. Mỗi câu "THE system SHALL X", hỏi: "Nếu tôi implement theo nghĩa đen nhất có thể, điều gì xảy ra?"

2

### Kỹ thuật 2 — Domain Expert Walk-through

Đọc spec cho người có domain expertise sâu (không phải developer). Domain expert thường ngay lập tức nhận ra business rule bị thiếu mà developer không nhìn thấy.

3

### Kỹ thuật 3 — Pre-mortem

Tưởng tượng ngày hôm nay là 3 tháng sau khi launch. Feature đã gây ra incident production nghiêm trọng. Viết ít nhất 3 scenario "spec failure" trước khi lock spec.

4

### Kỹ thuật 4 — AI Adversarial Review

Yêu cầu AI đóng vai developer muốn tạo code sai nhất có thể mà vẫn technically correct. Đây là "hardest test" cho spec của bạn.

## Ví dụ Devil's Advocate Review

```
# Spec: "THE system SHALL validate user age"
# Devil's Advocate: validate thế nào? >= 18? Phải là integer?
# Input "200" có valid không?
# Fix: "THE system SHALL validate age: integer,  $18 \leq \text{age} \leq 120$ "

# Spec: "THE system SHALL send confirmation email"
# Devil's Advocate: gửi khi nào? Bất đồng bộ hay đồng bộ?
# Retry nếu fail? Nếu email không tồn tại thì sao?
# Fix: "WHEN order confirmed, THE system SHALL asynchronously send
#       email to user (retry: 3 attempts, 60s apart).
#       Email delivery failure SHALL NOT block order confirmation."

# Spec: "THE system SHALL display error message"
# Devil's Advocate: message nào? Cho user hay cho developer?
# Tiếng Việt hay tiếng Anh? Có include error code không?
# Fix: specify exact message, language, user-facing vs technical detail
```

📌 **Quy tắc vàng:** Nếu một câu trong spec có thể được implement theo nhiều hơn 1 cách mà tất cả đều "technically correct" — câu đó chưa đủ cụ thể. Viết lại.

## Prompt: "Hardest Test" cho Spec

Tôi muốn bạn đóng vai một developer MUỐN làm đúng spec nhưng MUỐN tạo ra code sai nhất có thể mà vẫn technically correct.

Đọc spec này và liệt kê:

1. 5 cách implement "đúng spec" nhưng SAI business intent
2. 3 edge cases mà spec không cover rõ ràng, cho phép behavior tùy ý
3. 2 invariants bị thiếu mà nếu missing sẽ gây ra security issue

Với mỗi item, chỉ ra line/section trong spec cần update.

--- SPEC ---

[Paste spec]

--- END ---

# Đây là "hardest test" cho spec của bạn.

# Nếu AI tìm được NHIỀU → spec cần serious revision.

# Nếu AI tìm được ÍT → spec tương đối robust.

# Chạy prompt này TRƯỚC KHI lock spec, không phải sau.

 **Warning:** Nếu bạn cảm thấy không thoải mái khi chạy Adversarial Review vì "AI sẽ tìm ra quá nhiều vấn đề" — đó chính xác là lý do bạn phải chạy nó, và chạy trước khi code bắt đầu.

# Khi nào "Start Over" thay vì "Fix"

| Tình huống                                       | Fix hay Rewrite? | Lý do                                             |
|--------------------------------------------------|------------------|---------------------------------------------------|
| < 20% code cần thay đổi, bugs ở behavior detail  | Fix code         | Structural foundation còn good                    |
| Domain model sai (wrong entities, relationships) | Rewrite          | Structural fix không khả thi                      |
| Security vulnerability trong architecture        | Rewrite          | Patch security trên foundation sai = whack-a-mole |
| > 50% tests fail sau spec correction             | Rewrite          | Spec đã thay đổi đến mức code không còn relevant  |
| Business logic incorrectly distributed           | Rewrite          | Refactoring distributed logic > rewrite           |

📌 **Rule of thumb — "The 40% Rule":** Nếu fixing bad spec generated code yêu cầu thay đổi > 40% codebase, rewrite từ corrected spec thường nhanh hơn và kết quả tốt hơn. Lý do: code được viết để implement bad spec thường có assumptions sai ngấm trong toàn bộ structure — không phải chỉ ở surface level. "Fixing" thường chỉ di chuyển bugs từ nơi này sang nơi khác.

# Tổng kết Chương 8

| Phần            | Thông điệp cốt lõi                                                                                  |
|-----------------|-----------------------------------------------------------------------------------------------------|
| 8.1 — Điểm mạnh | Reproducibility, quality gates, knowledge persistence, parallel execution là lợi ích thực, đo được  |
| 8.2 — Chỉ trích | Waterfall chỉ đúng khi dùng sai. Context blindness là thực và không giải quyết hoàn toàn được       |
| 8.3 — Ranh giới | Exploration/R&D, hackathon, personal projects ngắn, landing pages: bỏ qua SDD                       |
| 8.4 — Góc nhìn  | Pragmatic SDD: EARS + Clarification cho mọi thứ phức tạp. Full SDD cho rủi ro cao                   |
| 8.5 — Bad Spec  | Bad spec + good AI = perfect execution of wrong thing. Invest in spec quality over AI model quality |

"SDD không phải là câu trả lời cho câu hỏi 'làm sao tôi build phần mềm tốt hơn?'. Nó là câu trả lời cho câu hỏi cụ thể hơn: 'làm sao tôi và AI cộng tác hiệu quả để build phần mềm phức tạp một cách đáng tin cậy?'"